DSA0310 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2

N-gram Language Modelling and Part-of-Speech Tagging for a Public Transport Enquiry Helpline

Student Details	
Name	V.Lakshmi Prasad
Reg Number	192425179
Department	AI&ML
Course	DSA0310 – Natural Language Processing
Submission Date	_____

TASK 1 – Counting Words and Building the Vocabulary (3 Marks)

1.1 Types, tokens, lemmas and word forms

Term	Meaning	Example
Token	Every occurrence of a word in the corpus.	"bus bus" contains 2 tokens.
Type	A distinct vocabulary item after the stated normalisation.	"bus bus" contains 1 type.
Lemma	Dictionary base form representing a lexeme.	books → book; went → go.
Word form	A particular grammatical spelling of a lemma.	book, books, booked, booking.

A reproducible count must specify tokenisation. Recommended policy: convert ordinary words to lowercase; preserve original text and character offsets; split punctuation except punctuation internal to times, decimals and recognised abbreviations; treat "6:30" as one time token; keep "Vellore" as one token; map case variants "Vellore/vellore" to one normalised type; retain numerals as tokens; preserve meaningful disfluencies such as "uh" and "um" in a separate disfluency class; add <s> and </s> as sentence-boundary symbols for n-gram training but exclude them from ordinary lexical token totals.

1.2 Effect of tokenisation decisions

Input	Policy A	Policy B	Effect
Vellore / vellore	Lowercase: one type	Case-sensitive: two types	Case folding reduces vocabulary by one.
6:30	One time token	Split into 6, : and 30	Policy B adds two tokens and may create punctuation types.
bus-stop	One hyphenated token	bus + stop	Policy B adds one token occurrence and may alter type counts.
uh, bus	Keep "uh"	Remove disfluency	Removal reduces token count and changes local n-grams.

Illustrative corpus count only: 160,000 lexical tokens, 8,200 surface types, 6,700 lemmas and 7,600 distinct normalised word forms. If case-sensitive tokenisation is used, assume 8,450 types; if lowercase and hyphen splitting are used, assume 8,360 types. These example counts show the direction of change, not actual corpus statistics.

1.3 Zipf's law

Zipf's law states that the frequency f(r) of a word is approximately inversely proportional to its rank r: f(r) ≈ C/r^s, where C is a constant and s is often near 1. On a log-log plot, rank and frequency should form an approximately straight descending line.

For the transport domain, high-frequency words such as "to", "bus", "ticket", "from", "train", "time" and "fare" may occur more often than general-language expectations. Station names and route identifiers form a long tail. Polite phrases and repeated call-centre prompts also cause deviations. Plot log(rank) against log(frequency), fit log f = log C − s log r, and report the fitted slope and R^2 from the real corpus.

1.4 Unknown-word threshold and OOV

Minimum training count	Vocabulary size (illustrative)	Training tokens mapped to <UNK>
1	8,200	0.0%
2	5,100	1.4%
3	4,200	2.1%
5	3,300	3.3%

Map every training word whose count is below threshold k to <UNK>. Apply the same mapping to development and test data. Development OOV rate = number of development tokens absent from the training vocabulary / total development tokens × 100. Example: 2,400 OOV tokens in 16,000 development tokens gives 15%. OOV rate limits all later models because an unseen station name has no direct word probability or reliable tag emission.

Report actual type/token/lemma/word-form counts, Zipf slope, R^2, vocabulary sizes and OOV rate after processing the required corpus.

TASK 2 – Unsmoothed N-gram Models (4 Marks)

2.1 Chain rule and Markov assumptions

For a word sequence W = w1…wn, the chain rule is $P(W) = \prod_i P(w_i \mid w1,\ldots,w_{i-1})$. A unigram model assumes each word is independent of its history: $P(w_i \mid \text{history}) \approx P(w_i)$. A bigram model keeps only the immediately previous word: $P(w_i \mid \text{history}) \approx P(w_i \mid w_{i-1})$. A trigram model keeps only the previous two words: $P(w_i \mid \text{history}) \approx P(w_i \mid w_{i-2},w_{i-1})$. Longer-range syntax, topic and discourse are ignored.

2.2 Maximum-likelihood estimates

Unigram: P(w) = C(w) / N

Bigram: P(w_i | w_(i-1)) = C(w_(i-1), w_i) / C(w_(i-1))

Trigram: P(w_i | w_(i-2),w_(i-1)) =

C(w_(i-2),w_(i-1),w_i) / C(w_(i-2),w_(i-1))

Add sentence boundaries consistently. For a sequence "<s> two tickets to Vellore </s>", the bigram factors are P(two|<s>)P(tickets|two)P(to|tickets)P(Vellore|to)P(</s>|Vellore). The trigram model uses the two preceding tokens for each prediction.

2.3 Hand calculation with a miniature training count table

Suppose the toy training corpus contains 100 tokens and the following counts: C(two)=8, C(tickets)=10, C(to)=20, C(Vellore)=4; C(two,tickets)=6, C(tickets,to)=8, C(to,Vellore)=3; C(<s>,two)=5. These are miniature demonstration counts.

P(two) = 8/100 = 0.08

P(tickets | two) = 6/8 = 0.75

P(to | tickets) = 8/10 = 0.80

P(Vellore | to) = 3/20 = 0.15

P(</s> | Vellore) = 2/4 = 0.50

Bigram sentence probability (using the four shown factors and EOS):

0.75 × 0.80 × 0.15 × 0.50 = 0.045

A perfectly reasonable sentence such as "please reserve a seat to Vellore" can receive probability zero if the training corpus never contained the bigram "reserve a". Under an unsmoothed bigram model C(reserve,a)=0, so P(a|reserve)=0 and the full product becomes zero.

2.4 Sparsity and generation

Bigram sparsity = 1 − (number of observed distinct bigrams / $|V|^2$). Trigram sparsity = 1 − (number of observed distinct trigrams / $|V|^3$). Also report unseen development n-gram rate = unseen development n-gram occurrences / all development n-gram occurrences.

Model	Illustrative observed fraction	Typical generation behaviour
Unigram	Not applicable to contexts	Frequent words appear, but order is mostly incoherent.
Bigram	0.8% of possible bigrams	Local phrases such as "bus to" emerge, but long structure is weak.
Trigram	0.03% of possible trigrams	More locally fluent phrases, but many contexts are unseen and

		generation can get stuck.

Generate ten samples per model by repeatedly sampling from the relevant empirical distribution. Stop at </s> or a maximum length. Higher order improves local fluency but increases sparsity, memory use and zero-count events.

TASK 3 – Smoothing and Discounting (5 Marks)

3.1 Why smoothing is needed

A zero count does not prove that a word sequence is impossible; it often means only that it was absent from a finite training sample. A valid conditional distribution must have non-negative probabilities and sum to one over all possible next words for each context.

3.2 Add-one and add-k smoothing

Add-one: $P_{Laplace}(w|h) = (C(h,w)+1) / (C(h)+V)$

Add-k: $P_{add\text{-}k}(w|h) = (C(h,w)+k) / (C(h)+kV)$

Toy context h = "to", vocabulary V=100, context count C(to)=20. Suppose observed continuations: bus=8, Vellore=3, Chennai=2, others total=7.

Continuation	Raw count	Add-one probability	Add-0.1 probability
bus	8	9/120 = 0.0750	8.1/30 = 0.2700
Vellore	3	4/120 = 0.0333	3.1/30 = 0.1033
Chennai	2	3/120 = 0.0250	2.1/30 = 0.0700
Other seen (total)	7	8/120 = 0.0667	7.1/30 = 0.2367
All unseen types (97)	0	97/120 = 0.8083	9.7/30 = 0.3233

Add-one changes the total denominator from 20 to 120 and reserves 100 pseudo-counts for the vocabulary. It takes substantial mass away from observed continuations. Add-k with k=0.1 reserves only 10 pseudo-counts and is less aggressive. The table groups unseen words; each individual unseen word receives the same add-k numerator.

3.3 Good–Turing discounting

Let N_r be the number of n-gram types occurring exactly r times. Good–Turing replaces a count r by $r^* = (r+1)N_{r+1}/N_r$. The total probability mass assigned to unseen n-grams is approximately N_1/N, where N_1 is the number of singleton n-gram types and N is the total number of n-gram tokens.

r	N_r (toy example)	r*
1	400	2 × 180/400 = 0.90
2	180	3 × 90/180 = 1.50
3	90	4 × 55/90 = 2.44
4	55	5 × 30/55 = 2.73

If there are 10,000 bigram tokens and 400 singleton bigram types, the estimated unseen mass is 400/10,000 = 0.04. In practice, raw Good–Turing estimates become unstable in the high-count tail because N_{r+1} may be zero or very small. Smooth the frequency-of-frequencies table or use a modified discounting method.

3.4 Comparison

Method	Strength	Weakness	Illustrative development perplexity
Unsmoothed	Faithful to observed counts	Zero probability for unseen events	Infinite
Add-one	Simple; all events non-zero	Severe discount with large V	210
Add-k (k=0.1)	Gentler smoothing	k must be selected on development data	145
Good–Turing	Uses singleton evidence to estimate unseen mass	Tail estimation needs care	128

These perplexities are example values only. A method should be chosen using a held-out development set, with parameters selected there and final test data untouched. Good–Turing or a modern discounting method is generally preferable to add-one for large vocabularies.

TASK 4 – Backoff and Deleted Interpolation (4 Marks)

Backoff uses a higher-order probability when reliable; if the n-gram is unseen or discounted, it backs off to a shorter context. Interpolation always combines several orders, even when the highest-order n-gram is observed.

Linear interpolation:
$P(w|u,v) = \lambda 3\ P(w|u,v) + \lambda 2\ P(w|v) + \lambda 1\ P(w)$
where $\lambda 1+\lambda 2+\lambda 3=1$ and each $\lambda \geq 0$.

Katz backoff:
$P_BO(w|h) = P_discounted(w|h)$, if seen
$= \alpha(h)\ P_BO(w|suffix(h))$, if unseen

Katz $\alpha(h)$ is a normalising weight that allocates the leftover probability mass to unseen continuations in proportion to the lower-order model. Without α, the seen and backed-off probabilities may not sum to one. Stupid backoff uses a fixed multiplier (commonly 0.4) when dropping an order; it produces a ranking score, not a normalised probability.

4.1 Deleted interpolation

Split part of training data as a held-out estimation set. For each trigram event, compare leave-one-out estimates at trigram, bigram and unigram levels; award its count to the order giving the highest estimate. Sum awarded counts and normalise to obtain λ weights. Do not use the final test set to tune λ.

Context count band	λ unigram	λ bigram	λ trigram
Low: 1–2	0.55	0.35	0.10
Medium: 3–10	0.25	0.50	0.25
High: >10	0.10	0.25	0.65

The table is illustrative. Low-count contexts have unreliable trigram estimates, so more weight is assigned to the unigram and bigram models. High-count contexts have enough observations to trust trigram detail.

4.2 Worked example

For a context "to Vellore", assume $P(w|to,Vellore)=0$ because the trigram was not observed, $P(w|Vellore)=0.10$ and $P(w)=0.02$. With low-count weights (0.10, 0.35, 0.55), interpolation gives $0.10\times0 + 0.35\times0.10 + 0.55\times0.02 = 0.046$. Backoff instead uses the lower-order distribution multiplied by α for the unseen trigram.

Model	Illustrative held-out perplexity	Treatment of unseen trigram
Katz backoff	112	Uses lower-order probability scaled by α.
Stupid backoff	Not a true perplexity model	Returns a score; cannot directly compute valid perplexity.
Linear interpolation	105	Always combines trigram, bigram and unigram.

Deleted interpolation	98	Uses held-out-estimated weights; often balances sparse and frequent contexts.

A hand trace for a real utterance should list each token’s trigram count, Katz α/backoff path, the three interpolated component probabilities and the final sequence log probability. Replace these illustrative results with measured values.

TASK 5 – Entropy, Cross-Entropy and Perplexity (4 Marks)

Entropy: $H(P) = -\Sigma x\ P(x) \log_2 P(x)$

Cross-entropy: $H(P,Q) = -\Sigma x\ P(x) \log_2 Q(x)$

Perplexity: $PP(Q) = 2^{[H(P,Q)]}$ (base-2 convention)

Entropy measures uncertainty in the true distribution. Cross-entropy measures the average number of bits required when the model Q predicts data generated by P. Perplexity is the exponential of cross-entropy and can be interpreted as the model's effective average number of choices per prediction. Perplexity 90 means the model is as uncertain, on average, as if it had 90 equally likely next-word choices; it does not mean 90% accuracy.

5.1 Hand calculations

Distribution A: P = (0.5, 0.5). $H = -0.5\log_2 0.5 - 0.5\log_2 0.5 = 1$ bit.

Distribution B: P = (0.9, 0.1). $H = -0.9\log_2 0.9 - 0.1\log_2 0.1 \approx 0.469$ bits.

Distribution C: P = (0.25, 0.25, 0.25, 0.25). H = 2 bits.

A nearly certain distribution has low entropy; a uniform distribution over more outcomes has higher entropy.

5.2 Sentence and corpus perplexity

For a sentence of N predicted tokens, cross-entropy = $-(1/N)\Sigma \log_2 P(w_i|\text{history})$; perplexity = 2^(cross-entropy). Use log probabilities to avoid numerical underflow. Include EOS consistently and document whether sentence-start symbols are predicted.

Model	Illustrative cross-entropy (bits/token)	Illustrative perplexity
Unigram	8.10	274.4
Bigram with smoothing	6.90	119.4
Trigram with interpolation	6.45	87.4

Compare models on the same untouched held-out set. Lower perplexity indicates better predictive fit, but it does not guarantee lower word error rate after speech-recognition rescoring.

5.3 Why domain matters

A transport corpus repeats route and service phrases, reducing uncertainty for common contexts. Rare station names, new route codes and code-mixed words increase uncertainty. Perplexity is comparable only when tokenisation, vocabulary, <UNK> handling and evaluation splits are identical.

TASK 6 – English Word Classes and Tagset Design (4 Marks)

English has open classes, which readily accept new words, and closed classes, whose membership changes slowly. For this domain, nouns include station and route names; verbs include book, cancel and return; adjectives include available and delayed; adverbs include tomorrow and nearby (depending on use). Closed classes include determiners, pronouns, prepositions, conjunctions, auxiliaries and particles.

6.1 Proposed 24-tag transport tagset

Tag	Meaning	Penn Treebank mapping
N	Common noun	NN/NNS
PROPN	Person/place/station/train name	NNP/NNPS
VERB	Main verb	VB/VBD/VBG/VBN/VBP/VBZ
AUX	Auxiliary verb	MD or VB* when auxiliary
ADJ	Adjective	JJ/JJR/JJS
ADV	Adverb	RB/RBR/RBS
PRON	Pronoun	PRP/PRP$
DET	Determiner	DT/PDT/WDT
ADP	Preposition/postposition	IN
CCONJ	Coordinating conjunction	CC
SCONJ	Subordinating conjunction	IN
NUM	Number/cardinal/ordinal	CD
PART	Particle/infinitival to	RP/TO
INTJ	Interjection/disfluency	UH
WH	Wh-word	WDT/WP/WP$/WRB
NEG	Negation	RB or separate domain tag
DATE	Date expression	CD/NN
TIME	Time expression	CD/NN
FARE	Fare/price amount	CD/$
ROUTE	Route/service identifier	NN/NNP
TICKET	Ticket/product category	NN
PUNCT	Punctuation	.,:; etc.
CM	Code-mixed or uncertain-language token	No direct PTB equivalent; map by contextual POS where possible
UNK	Unclassified token	No direct PTB equivalent; fallback

Domain-specific tags DATE, TIME, FARE, ROUTE and TICKET are semantic/syntactic hybrid labels. Preserve a secondary Penn Treebank POS label where possible. CM records language uncertainty; it should not replace the POS tag when a reliable POS is available.

6.2 Ambiguity examples

Word	Use 1	Use 2	Context clue
book	“book a ticket” – VERB	“the book” – N	Determiner and following noun/verb pattern.
return	“return tomorrow” – VERB	“return ticket” – ADJ/N modifier	Position and syntactic context.
stand	“stand here” – VERB	“bus stand” – N	Subject/imperative vs noun compound.
to	“to Chennai” – ADP	“to book” – PART	Following noun phrase vs base verb.

TASK 7 – Rule-Based Part-of-Speech Tagging (4 Marks)

A rule-based tagger assigns tags using a lexicon and ordered context rules. First assign likely tags from a dictionary; then apply rules that use suffixes, capitalisation, neighbouring words and sentence position.

7.1 Sample lexicon and rules

Word/pattern	Initial tag	Rule or context adjustment
book	N/VERB	After a pronoun or before determiner? Use "to book" / "please book" → VERB; after "the" → N.
return	N/VERB/ADJ	Before "ticket" → ADJ; after subject → VERB; "return fare" may be N modifier.
stand	N/VERB	After determiner → N; after subject or imperative start → VERB.
-ing	VBG/N	After auxiliary "is/are" → VBG; after determiner → often N.
-ed	VBD/VBN/JJ	After auxiliary "has" → VBN; before noun may be JJ.
Capitalised token	PROPN	At sentence start require evidence from gazetteer/context; do not blindly tag every initial capital as PROPN.

7.2 Ordered rules

1. If the previous tag is PART (to) and the current word is a known base verb, tag VERB.
2. If the previous word is a determiner (the/a/this) and the current word is a noun in the lexicon, tag N.
3. If the current word is in the station gazetteer, tag PROPN; retain a location entity label.
4. If the previous word is an auxiliary and the current word has an -ing form, tag VERB (VBG).
5. If the current word is a number followed by a currency unit, tag FARE.
6. If the word is a known time expression or matches a time pattern, tag TIME.

Rule conflict example: "the booking counter is open". A suffix rule may tag "booking" as VBG because it ends in -ing; a determiner-context rule identifies it as a noun. Give the local syntactic rule priority over the generic suffix rule. Ordered rules are understandable but become harder to maintain as exceptions accumulate.

Evaluate overall accuracy = correct tags / all tokens. Ambiguous-token accuracy = correct tags on pre-identified ambiguous tokens / number of ambiguous tokens. The second measure is especially important because easy function words can inflate overall accuracy while the system still fails on book/return/stand.

TASK 8 – Stochastic (HMM) Tagging and the Viterbi Algorithm (5 Marks)

8.1 HMM derivation

For words W and tags T, Bayes' rule gives $P(T|W) \propto P(W|T)P(T)$. The HMM factorises the tag sequence into first-order transitions and each word emission depends only on its current tag: $P(T,W) = P(t_1)\prod_{i=2}^{n}P(t_i|t_{i-1}) \times \prod_i P(w_i|t_i)$. Assumptions: (1) a tag depends only on the previous tag; (2) a word depends only on its own tag.

Transition: P(t_i | t_(i-1)) = C(t_(i-1),t_i) / C(t_(i-1))
Emission: P(w_i | t_i) = C(t_i,w_i) / C(t_i)
Viterbi:
δ_i(t) = max_s [δ_(i-1)(s) × P(t|s) × P(w_i|t)]

Apply add-k or Lidstone smoothing to transition and emission distributions. For every previous tag, verify that the sum of transition probabilities over all next tags is 1. Use an explicit UNK emission class and suffix/capitalisation features for unknown words.

8.2 Viterbi example

Sentence: "please book two tickets to Vellore". A simplified tag set is INTJ/VERB/NUM/N/ADP/PROPN. The table below is an illustrative trellis; scores are path probabilities using toy transition and emission probabilities, not corpus measurements.

Position / word	Candidate tag	Best score δ (illustrative)	Back-pointer
please	INTJ	0.20	START
please	VERB	0.05	START
book	VERB	0.12	INTJ
book	N	0.01	INTJ
two	NUM	0.072	VERB
two	N	0.004	VERB
tickets	N	0.0432	NUM
tickets	VERB	0.001	NUM
to	ADP	0.0302	N
to	PART	0.0021	N
Vellore	PROPN	0.0211	ADP
Vellore	N	0.0005	ADP

The best path is INTJ → VERB → NUM → N → ADP → PROPN. Recover it by following the stored back-pointers from the highest final score. The actual numerical path probability is the product of all selected initial, transition and emission terms; report it from your trained model.

8.3 Baselines, exhaustive search and unknown words

Most-frequent-tag baseline assigns the globally most frequent tag to every token (or the most frequent tag per word if a lexicon baseline is used). Exhaustive decoding evaluates every possible tag sequence: for n tokens and K tags, it considers K^n paths. Viterbi uses dynamic programming in $O(nK^2)$ time and $O(nK)$ trellis storage.

Decoder	Illustrative tagging accuracy	Operations for 6 tokens, K=6
Most-frequent tag	72%	About 6 assignments
Exhaustive search	Same optimum as Viterbi	$6^6 = 46{,}656$ sequences (with path scoring)
Viterbi	91%	About $nK^2 = 216$ transition comparisons

Unknown-word method: use suffix evidence (-ing, -ed, -ly), capitalisation and open-class priors. A capitalised unseen word at a destination slot may receive high PROPN probability. Evaluate accuracy separately on OOV tokens; do not hide poor OOV performance inside overall accuracy.

TASK 9 – Transformation-Based (Brill) Tagging (4 Marks)

Brill tagging begins with an initial tagger, commonly a most-frequent-tag baseline. It proposes candidate transformations from templates, applies each candidate to the development/training error set, selects the rule with the greatest net error reduction, and repeats until no rule improves the score or a maximum rule count is reached.

9.1 Templates

- Change tag A to B when the previous tag is X.
- Change tag A to B when the next tag is X.
- Change tag A to B when the previous word is W.
- Change tag A to B when the next word is W.
- Change tag A to B when the current word is W.
- Change tag A to B when the current word ends with suffix S.
- Change tag A to B when the previous two tags are X,Y.
- Change tag A to B when the current word is capitalised and is in a station gazetteer.

A ±1 word/tag window captures local syntactic cues at manageable computational cost. Wider windows can help route phrases but increase candidate rules and overfitting.

9.2 Example learned transformations

Order	Illustrative transformation	Errors corrected	Errors introduced
1	Tag “book” as VERB when next tag is DET or the previous word is “please”.	38	4
2	Tag a station-gazetteer word as PROPN.	31	2
3	Change N to VERB after PART “to”.	25	3
4	Change VBG to N after DET.	19	2
5	Change NN to NUM for digit strings.	16	0
6	Change N to ADJ before “ticket”.	13	2
7	Change NN to TIME when matching a time pattern.	11	0
8	Change VERB to N after DET.	9	2
9	Change N to PROPN	8	3

	when capitalised and preceded by “to”.		
10	Change JJ to VERB after modal AUX.	7	1

These rules are examples of the format, not rules learned from a real annotated dataset. A real run should print the first ten rules and corrected/introduced errors from the learner.

A linguistic generalisation is “to” + base form → VERB. A training artefact might be “after the token ‘Monday’, tag the next word as PROPN” if it only reflects a few examples. Readability is a strength: rules can be inspected directly, unlike a large table of HMM probabilities.

Plot accuracy against number of rules. Accuracy normally rises then plateaus; if development accuracy falls while training accuracy continues to rise, rules beyond that point overfit. Stop at the best development accuracy.

TASK 10 – Implement and Test the Models (3 Marks)

10.1 Python/NLTK implementation outline

```
from collections import Counter, defaultdict
import math, re, random, time
from nltk import word_tokenize
from nltk.stem import PorterStemmer

def count_ngrams(sentences, n):
    counts = Counter()
    for sent in sentences:
        padded = ['<s>']*(n-1) + sent + ['</s>']
        counts.update(tuple(padded[i:i+n])
                      for i in range(len(padded)-n+1))
    return counts

def mle_bigram(bigrams, unigrams, h, w):
    return bigrams[(h,w)] / unigrams[h] if unigrams[h] else 0.0

def add_k_bigram(c_hw, c_h, V, k=0.1):
    return (c_hw+k)/(c_h+k*V)

def perplexity(log_probs, n):
    return math.exp(-sum(log_probs)/n)

def viterbi(words, tags, start, trans, emit):
    # log-space dynamic programming; return best tag path
    dp, back = [{}], [{}]
    for t in tags:
        dp[0][t] = math.log(start[t]) + math.log(emit[t].get(words[0], 1e-9))
    for i in range(1, len(words)):
        dp.append({}); back.append({})
        for t in tags:
            candidates = [(dp[i-1][s] + math.log(trans[s][t]), s)
                          for s in tags]
            score, prev = max(candidates)
            dp[i][t] = score + math.log(emit[t].get(words[i], 1e-9))
            back[i][t] = prev
    last = max(tags, key=lambda t: dp[-1][t])
    path = [last]
    for i in range(len(words)-1, 0, -1):
        path.append(back[i][path[-1]])
    return list(reversed(path)), dp[-1][last]
```

Extend this core with vocabulary/UNK mapping, unigram and trigram counts, Good–Turing frequency-of-frequencies, Katz α, interpolation weight estimation, rule-based tagging and a Brill learner. Use only the standard library and NLTK as required. Fix a random seed (for example random.seed(2026)) and keep utterances separated across 80/10/10 splits.

10.2 Reproducible tests and reporting

Component	Test	Expected check
Unsmoothed bigram	Sentence containing unseen "reserve a"	Probability is exactly zero.
Interpolation	Check $\lambda_1+\lambda_2+\lambda_3$	Sum equals 1.0.
HMM	Check each transition row	Each row sums to 1 within rounding tolerance.
Viterbi	Six-word example	Backtracking returns the highest-scoring path.
Brill	First ten transformations	Re-running with same seed/data gives same rule order.
Perplexity	Same fixed test set	No zero probabilities after smoothing; finite score.

Record training time with time.perf_counter(), model size with a documented memory estimator or serialized size, perplexity on the same held-out tokens, tagging accuracy and tagging throughput (tokens/second). Include at least two plots: perplexity by n-gram order/smoothing and tagging accuracy by training-set size. The figures must be generated from actual runs.

TASK 11 – Compare, Recommend and Defend the Design (2 Marks)

Candidate	Advantages	Risks / costs
Interpolated trigram LM	Uses local context and lower-order evidence; finite probability for unseen events.	More memory than unigram/bigram; still weak on unseen names and long context.
Katz backoff LM	Uses higher-order evidence when present and lower-order model otherwise.	Requires discounting and normalisation; implementation is more complex.
Rule-based tagger	Transparent, fast and easy to audit.	Manual rule maintenance; brittle on unseen contexts.
HMM tagger	Sequence-level decisions; learns transition/emission patterns.	OOV emissions and independence assumptions can hurt accuracy.
Brill tagger	Readable corrections; can improve a baseline using context.	Training and rule search cost; can overfit.

Recommended baseline deployment: interpolated trigram language model with Good–Turing/discounted lower-order estimates, paired with an HMM tagger enhanced by a station gazetteer and suffix/capitalisation unknown-word handling. This combination balances contextual scoring, probability coverage and sequence tagging. If measured Brill accuracy is higher on the held-out set and maintenance is acceptable, it can be deployed as a correction layer.

Expected accuracy and WER must come from the actual test set. Typical failure modes include unseen station names, code-mixed words, noisy ASR substitutions, long-distance dependencies and ambiguous words without enough context. A caller may hear a wrong destination, fare or service option; therefore, uncertain entities should trigger confirmation rather than silent commitment.

Annotation effort: hand-tag at least 3,000 tokens and double-annotate 500. Report Cohen's κ or token agreement. Rules require expert review and regression tests. More annotated data is more valuable than more hand-written rules when errors are diverse, context-dependent and no longer fixed by a small number of reliable patterns.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design (8 Marks)

A. Impact analysis (2 Marks)

The new metro/intercity domain adds hundreds of station names, train names and fare categories. Code-mixed utterances such as "Chennai to Madurai bus timing enna" introduce words and grammatical patterns absent from English-only training. More than 15% of tokens are OOV. The original assumptions of a mostly closed vocabulary, stable OOV rate, English-only word classes and domain-specific transition probabilities no longer hold.

- Station names are the most damaging OOVs because they identify the caller's origin/destination; losing them can reverse the meaning of the enquiry.
- Unknown words cause low or fallback emission probabilities in the HMM and high perplexity in the language model.
- English-only tag categories cannot represent code-mixed tokens or language-specific particles reliably.
- Old transition probabilities are biased toward bus enquiries and may mis-tag rail-service phrases.

If unknown-word accuracy measured in Task 8 is A and OOV share is 0.15, a simple upper-level approximation for overall accuracy is (0.85 × known-word accuracy) + (0.15 × OOV accuracy), assuming the remaining known-token accuracy is unchanged. For example, known accuracy 94% and OOV accuracy 40% gives 85.9% overall. This is an illustration; actual degradation depends on context and OOV distribution. Perplexity will rise because unseen words receive only fallback probability.

B. Evaluation of the original design (2 Marks)

New utterance	Likely original-model failure	First component affected
"two tickets to Koyambedu" (unseen station)	Station token mapped to <UNK>; destination identity lost.	Vocabulary / language model; then HMM emission.
"Chennai to Madurai bus timing enna"	English words may be tagged; "enna" may be UNK or incorrectly tagged as noun.	English-only lexicon and tagset.
"train to Poonamallee tomorrow varuma"	Station and code-mixed verb may be unknown; model may prefer a familiar bus phrase.	LM OOV handling and HMM emissions/transitions.

Measure degradation on a separately labelled new-domain sample: compute perplexity with the same tokenisation and UNK policy, WER after five-hypothesis rescoring, and POS accuracy overall and on OOV/code-mixed tokens. The language model's lexical coverage and the HMM's unknown-word emissions are likely to degrade fastest. A station gazetteer is economical for proper nouns; a character-level model helps spelling variants; class-based n-grams share statistics across station-name slots.

C. Adaptation or redesign (2 Marks)

Decision: MODIFY the original design. Retain the n-gram framework and HMM where useful, but add open-vocabulary and multilingual support.

7. Maintain a dynamic station/train gazetteer with canonical name, aliases, language/script variants, service type and location metadata. Map aliases to a canonical entity ID.
8. Replace a single word vocabulary with word classes: <STATION>, <TRAIN>, <FARE>, <TIME>, <ROUTE>. Train class-based n-grams so unseen station names can still occupy a station slot.
9. Add character n-gram or subword scoring to rank phonetic/OCR variants of unseen names. Keep original ASR hypotheses and do not silently overwrite uncertain names.
10. Re-estimate Good–Turing/discounting statistics and deleted-interpolation λ weights using a representative mixed-domain development set. Keep a separate untouched test set.
11. Extend the tagset with CM (code-mixed token), LANG-specific particles where useful, and an explicit PROPN subtype for station/train names. Preserve the Penn Treebank mapping as a secondary label.
12. Add a multilingual/open-vocabulary HMM emission model using suffixes, character features, language ID and gazetteer membership. Retrain transitions with bus, metro and rail utterances.
13. Refresh the gazetteer weekly or whenever new stations/services are announced; retrain or incrementally update the language/tag models monthly, with a full evaluation before deployment.

Expected effects: OOV rate should fall for known station entities through gazetteer aliases and class mapping; character models should improve spelling robustness; code-mixed tagging accuracy should improve after annotation. Runtime and annotation effort increase. Quantify the improvement by evaluating old and modified models on the same new-domain test set; do not invent an improvement percentage.

D. Evidence-based defence (2 Marks)

The modification follows directly from the failure analysis. The OOV tokens carry critical intent, so treating them as a generic <UNK> loses information. A gazetteer and entity classes preserve the station role even when the exact surface name is new. Character n-grams address phonetic spelling variation, while code-mixed tags and multilingual emissions address the mismatch in word classes. Re-estimating interpolation weights prevents the old bus-heavy distribution from dominating rail enquiries.

An n-gram model cannot capture arbitrary long-distance dependencies, full discourse meaning, or all possible unseen word sequences. Even with unlimited data, a fixed-order Markov model only conditions on a fixed number of preceding tokens. It also cannot reliably infer the meaning of a new station name without external knowledge. The final system therefore combines statistical language modelling with entity resources, multilingual features and caller confirmation for uncertain destinations.

Conclusion

A transport helpline requires more than word-frequency counting. A smoothed and interpolated n-gram model can rescore ASR hypotheses, while a tagger resolves context-sensitive word classes before entity extraction. The best design is measured on held-out data and adapted as the service expands. Under the changed metro/rail and code-mixed conditions, a modified hybrid system with station gazetteers, class-based n-grams, character-level unknown-word handling and an expanded tagger is more suitable than retaining the original English-only model unchanged.

Key formulas and glossary

Item	Definition
Type/token	Distinct vocabulary item / each occurrence.
OOV rate	Unseen development/test tokens ÷ total tokens.
MLE	Maximum likelihood estimate from relative frequency.
Entropy	Uncertainty in a probability distribution.
Cross-entropy	Average negative log probability assigned by model to true data.
Perplexity	Exponential of cross-entropy.
Backoff	Use shorter context when higher-order evidence is missing.
Interpolation	Weighted combination of multiple n-gram orders.
HMM	Hidden Markov Model for tag sequence and word emissions.
Viterbi	Dynamic programming for the highest-probability tag path.
Brill tagging	Greedy transformation-based correction of an initial tag sequence.
OOV	Out-of-vocabulary word.